LAB SHEET 5.1

First Steps with Mininet: Topologies, CLI, and Traffic Capture

Week 5	
Duration	2 hours
Deliverable	Screenshot report (PDF) + terminal transcript, submitted via GitHub repository

1. Learning Objectives

By the end of this lab, you should be able to:

1. Launch Mininet with different built-in topologies directly from the command line using the mn tool.
2. Use core Mininet CLI commands to inspect and test a running topology (nodes, net, dump, pingall, iperf, link).
3. Capture and inspect real traffic on a Mininet virtual link using tcpdump/Wireshark.
4. Compare the structure and behavior of single, linear, and tree default topologies.
5. Explain, from hands-on experience, when to reach for Mininet-style emulation versus a pure simulator such as NS-3 or OMNeT++.

Prerequisites

- Lab 1 completed: working Mininet installation, verified with sudo mn --test pingall.
- Comfort navigating a Linux terminal and reading basic networking output (IP addresses, interface names).

Software Materials

- Mininet (installed in Lab 1)
- iperf/iperf3 (installed alongside Mininet)
- tcpdump, and optionally Wireshark with a graphical display for Part F

Lab Tasks

Complete the following tasks and record evidence (terminal output / screenshots) for each in your report.

1. **Task 1:** Screenshot of the single,3 topology's pingall showing 0% dropped.

```
osboxes@osboxes:~/netprog-2026$ sudo mn --topo single,3
[sudo] password for osboxes:
/usr/local/bin/mn:4: DeprecationWarning: pkg_resources is deprecated as an API.
See https://setuptools.pypa.io/en/latest/pkg_resources.html
  __import__('pkg_resources').run_script('mininet-wifi==2.7', 'mn')
```

```
osboxes@osboxes: ~/netprog-2026
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet-wifi> nodes
available nodes are:
c0 h1 h2 h3 s1
mininet-wifi> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0
c0
mininet-wifi> dump
<Host h1: h1-eth0:10.0.0.1 pid=19341>
<Host h2: h2-eth0:10.0.0.2 pid=19343>
<Host h3: h3-eth0:10.0.0.3 pid=19345>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None pid=19350>
<Controller c0: 127.0.0.1:6653 pid=19334>
mininet-wifi> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet-wifi>
```

2. **Task 2:** Screenshot of the iperf result from Part B, plus the h1-h3 ping RTT. Report the measured bandwidth and average RTT.

```
mininet-wifi> iperf
*** Iperf: testing TCP bandwidth between h1 and h3
*** Results: ['12.3 Gbits/sec', '12.2 Gbits/sec']
mininet-wifi> h1 ping -c 5 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=25.6 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.336 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.101 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.073 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=0.067 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4004ms
rtt min/avg/max/mdev = 0.067/5.238/25.615/10.188 ms
mininet-wifi>
```

TCP Bandwidth between h1 and h3: 12.3 Gbits/sec and 12.2 Gbits/sec.

Round-Trip Time: min = 0.067ms, average = 5.238 ms, max = 25.615 ms, mean deviation = 10.188 ms

3. **Task 3:** Screenshot of linear,4's net output and pingall result. In 1-2 sentences, describe how this topology's structure differs from single,3's.

```
osboxes@osboxes:~/netprog-2026$ sudo mn --topo linear,4
/usr/local/bin/mn:4: DeprecationWarning: pkg_resources is deprecated as an API. See https:/
/setuptools.pypa.io/en/latest/pkg_resources.html
  __import__('pkg_resources').run_script('mininet-wifi==2.7', 'mn')
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s2, s1) (s3, s2) (s4, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet-wifi> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth1
h3 h3-eth0:s3-eth1
h4 h4-eth0:s4-eth1
s1 lo:  s1-eth1:h1-eth0 s1-eth2:s2-eth2
s2 lo:  s2-eth1:h2-eth0 s2-eth2:s1-eth2 s2-eth3:s3-eth2
s3 lo:  s3-eth1:h3-eth0 s3-eth2:s2-eth3 s3-eth3:s4-eth2
s4 lo:  s4-eth1:h4-eth0 s4-eth2:s3-eth3
c0
```

```
mininet-wifi> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet-wifi> h1 ping -c 5 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=0.454 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=0.086 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.065 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=0.093 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=0.075 ms

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4081ms
rtt min/avg/max/mdev = 0.065/0.154/0.454/0.150 ms
```

Round-Trip Time: min = 0.065 ms, average = 0.154 ms, max = 0.454 ms, mean deviation = 0.150 ms

Answer In single,3, there is only one switch (s1), and all three hosts connect directly to it, so any two hosts can reach each other by passing through just one switch. In linear,4, there are four switches (s1 to s4) connected one after another in a line, and each switch has only one host. This means traffic between hosts at opposite ends, like h1 and h4, must travel through all four switches, making the path longer than in single,3.

4. **Task 4:** Screenshot of tree,depth=2,fanout=2's net output. Report the actual number of hosts and switches you observed, and show that it matches (or explains any mismatch with) the formulas hosts = fanout^depth and switches = (fanout^depth - 1) / (fanout - 1). Then repeat this for a second depth/fanout combination of your own choosing: show your predicted counts (calculated before launching it), a screenshot of its net output, and confirm whether your prediction matched.

tree,depth=2,fanout=2's net output:

```
osboxes@osboxes:~/netprog-2026$ sudo mn --topo tree,depth=2,fanout=2
[sudo] password for osboxes:
/usr/local/bin/mn:4: DeprecationWarning: pkg_resources is deprecated as an API. See https:/
/setuptools.pypa.io/en/latest/pkg_resources.html
  __import__('pkg_resources').run_script('mininet-wifi==2.7', 'mn')
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3
*** Adding links:
(s1, s2) (s1, s3) (s2, h1) (s2, h2) (s3, h3) (s3, h4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 3 switches
s1 s2 s3
*** Starting CLI:
mininet-wifi> net
h1 h1-eth0:s2-eth1
h2 h2-eth0:s2-eth2
h3 h3-eth0:s3-eth1
h4 h4-eth0:s3-eth2
s1 lo:  s1-eth1:s2-eth3 s1-eth2:s3-eth3
s2 lo:  s2-eth1:h1-eth0 s2-eth2:h2-eth0 s2-eth3:s1-eth1
s3 lo:  s3-eth1:h3-eth0 s3-eth2:h4-eth0 s3-eth3:s1-eth2
c0
mininet-wifi> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
```

hosts = 2^2 = **4** switches = $(2^2 - 1) / (2 - 1) = 3 / 1 =$ **3**

Output of net = 4 hosts (h1–h4) and 3 switches (s1–s3).

tree,depth=3,fanout=2's net output:

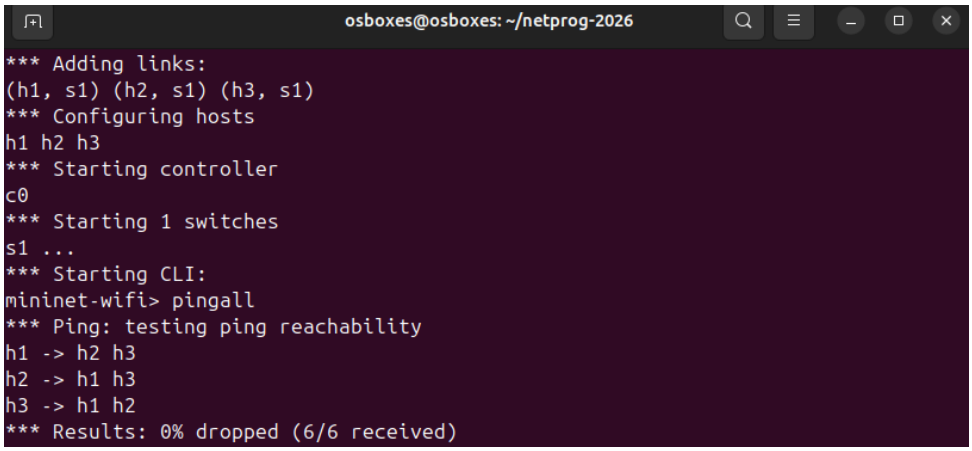
```
osboxes@osboxes:~/netprog-2026$ sudo mn --topo tree,depth=3,fanout=2
[sudo] password for osboxes:
/usr/local/bin/mn:4: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.
io/en/latest/pkg_resources.html
  __import__('pkg_resources').run_script('mininet-wifi==2.7', 'mn')
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5 h6 h7 h8
*** Adding switches:
s1 s2 s3 s4 s5 s6 s7
*** Adding links:
(s1, s2) (s1, s5) (s2, s3) (s2, s4) (s3, h1) (s3, h2) (s4, h3) (s4, h4) (s5, s6) (s5, s7) (s6, h5) (s6, h6)
(s7, h7) (s7, h8)
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8
*** Starting controller
c0
*** Starting 7 switches
s1 s2 s3 s4 s5 s6 s7 ...
*** Starting CLI:
mininet-wifi> net
h1 h1-eth0:s3-eth1
h2 h2-eth0:s3-eth2
h3 h3-eth0:s4-eth1
h4 h4-eth0:s4-eth2
h5 h5-eth0:s6-eth1
h6 h6-eth0:s6-eth2
h7 h7-eth0:s7-eth1
h8 h8-eth0:s7-eth2
s1 lo:  s1-eth1:s2-eth3 s1-eth2:s5-eth3
s2 lo:  s2-eth1:s3-eth3 s2-eth2:s4-eth3 s2-eth3:s1-eth1
s3 lo:  s3-eth1:h1-eth0 s3-eth2:h2-eth0 s3-eth3:s2-eth1
s4 lo:  s4-eth1:h3-eth0 s4-eth2:h4-eth0 s4-eth3:s2-eth2
s5 lo:  s5-eth1:s6-eth3 s5-eth2:s7-eth3 s5-eth3:s1-eth2
s6 lo:  s6-eth1:h5-eth0 s6-eth2:h6-eth0 s6-eth3:s5-eth1
s7 lo:  s7-eth1:h7-eth0 s7-eth2:h8-eth0 s7-eth3:s5-eth2
c0
mininet-wifi> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
```

hosts = 2^3 = **8** switches = $(2^3 - 1) / (2 - 1) = 7 / 1 =$ **7**

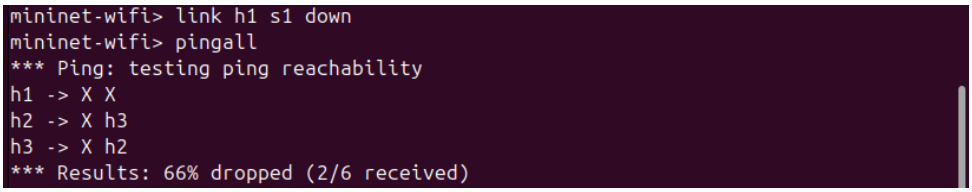
Output of net = 8 hosts (h1–h8) and 7 switches (s1–s7).

5. **Task 5:** Screenshots of Part E's three pingall results (before the link failure, during the failure, and after recovery), clearly showing the affected host failing and then recovering.

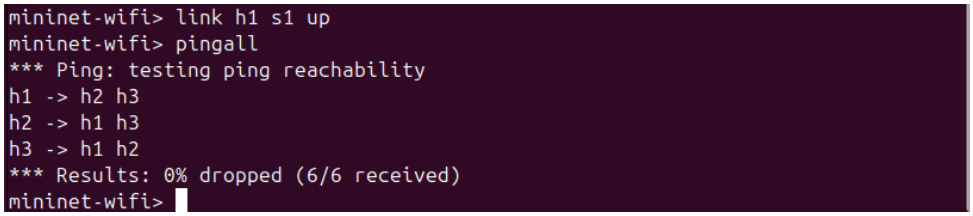
Before the link failure:



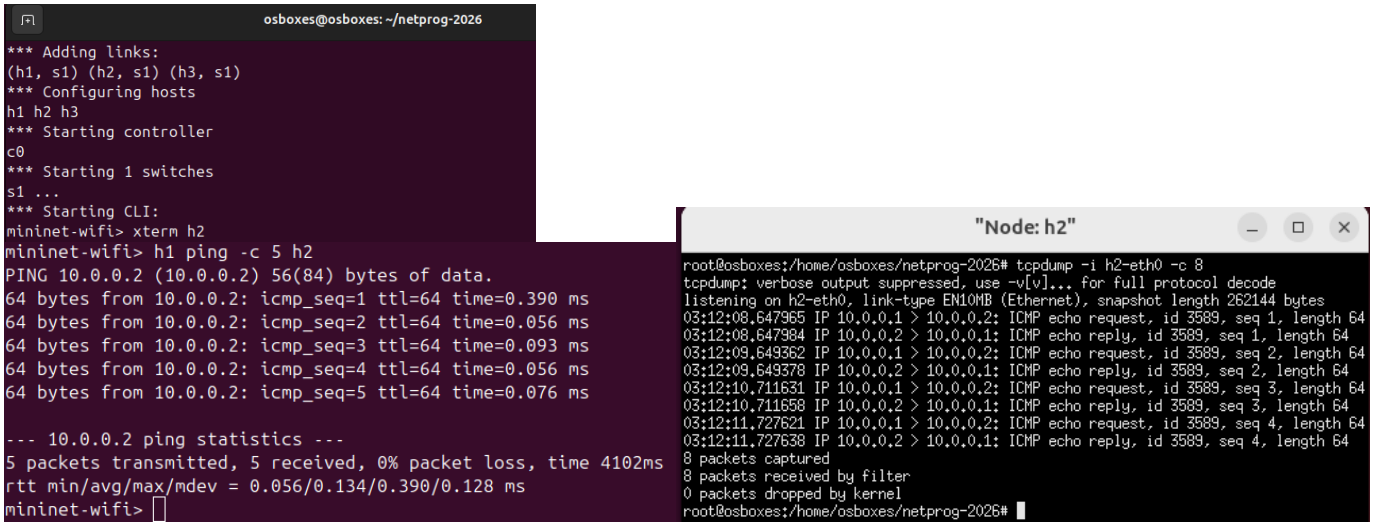
During the failure:



After recovery:



6. **Task 6:** Screenshot of the tcpdump capture from Part F showing real ICMP echo request/reply frames. In 2-3 sentences, connect this observation back to the simulation-vs-emulation distinction from Lab 1.



Answer The tcpdump output shows real ICMP echo request and reply packets moving across h2's network interface. This is emulation: Mininet runs real network code and sends real packets between hosts, instead of just calculating results from a model the way a simulator like NS-3 does. Because of that, I could capture the traffic with an ordinary tool like tcpdump, just like on a physical network.

Reflection Questions

Answer briefly (2-4 sentences each) in your report:

1. Compare the linear and tree topologies in terms of how many switch hops separate the two farthest-apart hosts as the network grows larger. Which topology scale is better, and why?

   Answer In a linear topology, the farthest hosts must pass through every switch in the chain, so the number of hops grows directly with the network size. With 4 switches the path crosses 4 switches, and with 100 switches it would cross all 100. In a tree topology, traffic only goes up to the root and back down, so the path crosses at most 2 × depth − 1 switches, which grows much more slowly as hosts are added. The tree scales better because it can connect many more hosts while keeping the path between any two of them short.

2. Why might the average ping RTT you measured in Part C (linear,4, host to host across the whole chain) be higher than the RTT you measured in Part A (single,3)?

   Answer In single,3, a ping from h1 to h3 passes through only one switch, but in linear,4 a ping from h1 to h4 has to pass through four switches each way. Every switch adds a small processing delay, so the delay adds up across the chain. The first ping is usually much slower too, because each switch must ask the controller where to send the packet before a flow rule is installed. That one slow ping raises the average more in linear,4, since there are more switches involved.

3. Based on today's hands-on experience, describe one scenario where you would choose Mininet-style emulation for a project, and one scenario where a pure simulator like NS-3 or OMNeT++ would be the better choice instead.

   Answer I would choose Mininet if I wanted to test a real program, such as the UDP chat from Lab 4 or an SDN controller, because Mininet runs the actual code and sends real packets that I can capture with tcpdump. A simulator like NS-3 or OMNeT++ would be better for studying a very large network with thousands of nodes, which would be too much for one computer to emulate. A simulator also gives repeatable, precisely timed results, while Mininet's timing depends on how busy the host machine is.